

Deployment Guide - MorphoScan Pro

Overview

This guide provides step-by-step instructions for deploying MorphoScan Pro to production environments. The application is optimized for deployment on Vercel, Netlify, or any static hosting platform with edge function support.

Table of Contents

1. [Prerequisites](#)
 2. [Environment Configuration](#)
 3. [Build Variants](#)
 4. [Deployment Platforms](#)
 5. [Database Setup](#)
 6. [Edge Functions](#)
 7. [Post-Deployment Verification](#)
 8. [Monitoring & Logging](#)
 9. [Rollback Procedures](#)
 10. [Troubleshooting](#)
-

Prerequisites

System Requirements

- **Node.js:** v22.x or higher
- **npm:** v10.x or higher
- **Git:** Latest version
- **Supabase CLI:** v1.x (for database migrations)

Required Accounts

-  Supabase account (database & auth)
-  Stripe account (payment processing)
-  Vercel/Netlify account (hosting)
-  Sentry account (error tracking - optional but recommended)

Required Access

- Database admin credentials
 - Stripe API keys (live mode)
 - Deployment platform credentials
 - Domain DNS management (for custom domains)
-

Environment Configuration

Environment Variables Checklist

Create a `.env.production` file or configure environment variables in your hosting platform:

1. Supabase Configuration (Required)

```
# Supabase Project URL
VITE_SUPABASE_URL=https://your-project.supabase.co

# Supabase Publishable Key (anon key)
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

# Supabase Project Reference ID
VITE_SUPABASE_PROJECT_ID=your-project-ref
```

2. Application Configuration (Required)

```
# Application Version: 'sfw', 'nsfw', or 'hybrid'
VITE_APP_VERSION=hybrid

# Distribution Channel: 'store' or 'direct'
VITE_DISTRIBUTION_CHANNEL=direct

# Environment: 'production', 'staging', 'development'
VITE_APP_ENVIRONMENT=production
```

3. Security Configuration (Required)

```
# Client-side encryption salt (generate unique value)
VITE_CLIENT_ENCRYPTION_SALT=your-unique-salt-32-chars-min

# Support email for security issues
VITE_SECURITY_EMAIL=security@morphoscan.com

# DLC support email
VITE_DLC_SUPPORT_EMAIL=dlc-support@morphoscan.com
```

4. Stripe Configuration (Required for payments)

```
# Stripe Publishable Key (live mode)
VITE_STRIPE_PUBLISHABLE_KEY=pk_live...

# Stripe Price IDs for each product tier
VITE_STRIPE_PRICE_ID_BASIC_MONTHLY=price...
VITE_STRIPE_PRICE_ID_BASIC_YEARLY=price...
VITE_STRIPE_PRICE_ID_PRO_MONTHLY=price...
VITE_STRIPE_PRICE_ID_PRO_YEARLY=price...
VITE_STRIPE_PRICE_ID_PREMIUM_MONTHLY=price...
VITE_STRIPE_PRICE_ID_PREMIUM_YEARLY=price...

# DLC Package Price IDs
VITE_STRIPE_PRICE_ID_NSFW_BASIC=price...
VITE_STRIPE_PRICE_ID_NSFW_PREMIUM=price...
VITE_STRIPE_PRICE_ID_3D_MODEL_VIEWER=price...
# ... (20+ more DLC price IDs - see .env.example)
```

5. Sentry Configuration (Optional but recommended)

```
# Sentry DSN for error tracking
VITE_SENTRY_DSN=https://...@sentry.io/...

# Sentry Environment
VITE_SENTRY_ENVIRONMENT=production

# Sentry Trace Sample Rate (0.0 to 1.0)
VITE_SENTRY_TRACES_SAMPLE_RATE=0.1
```

Generating Secure Values

Encryption Salt

```
# Generate a secure random salt (32+ characters)
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Verifying Configuration

```
# Run production readiness check
npm run check:production
```

Build Variants

MorphoScan Pro supports multiple build variants for different distribution channels:

Available Build Commands

```
# Standard production build (uses env variables)
npm run build

# Development build (with debug info)
npm run build:dev

# Version-specific builds
npm run build:sfw          # Safe-for-work version
npm run build:nsfw         # NSFW version (adult content)
npm run build:hybrid        # Hybrid version (DLC-based content)

# Channel-specific builds
npm run build:sfw:store    # SFW for app stores
npm run build:sfw:direct   # SFW for direct distribution
npm run build:nsfw:direct  # NSFW for direct distribution
npm run build:hybrid:store  # Hybrid for app stores
npm run build:hybrid:direct # Hybrid for direct distribution
```

Build Variants Comparison

Variant	Use Case	Content	App Store
SFW	General audience	Health tracking only	✅ Yes
NSFW	Adult audience	Full NSFW features	❌ No
Hybrid	Flexible	Base + DLC purchases	✅ Yes

Recommended Variant

Use **hybrid** for maximum flexibility:

- Approved for app stores
- Users can purchase NSFW DLC separately
- Best monetization potential

Deployment Platforms

Option 1: Vercel (Recommended)

Why Vercel?

- ✅ Excellent Next.js/Vite support
- ✅ Edge functions for dynamic content
- ✅ Automatic HTTPS
- ✅ Global CDN
- ✅ Free tier available

Deployment Steps

1. Install Vercel CLI

```
npm install -g vercel
```

1. Login to Vercel

```
vercel login
```

1. Configure Project

Create `vercel.json`:

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "framework": "vite",
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ],
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        {
          "key": "X-Frame-Options",
          "value": "DENY"
        },
        {
          "key": "X-Content-Type-Options",
          "value": "nosniff"
        },
        {
          "key": "X-XSS-Protection",
          "value": "1; mode=block"
        },
        {
          "key": "Strict-Transport-Security",
          "value": "max-age=31536000; includeSubDomains"
        }
      ]
    }
  ]
}
```

1. Deploy

```
# Deploy to preview
vercel

# Deploy to production
vercel --prod
```

1. Set Environment Variables

```
# Via CLI
vercel env add VITE_SUPABASE_URL production
vercel env add VITE_SUPABASE_ANON_KEY production
# ... add all required variables

# Or via Vercel Dashboard:
# Settings > Environment Variables
```

Option 2: Netlify

Deployment Steps

1. Install Netlify CLI

```
npm install -g netlify-cli
```

1. Login to Netlify

```
netlify login
```

1. Configure Project

Create `netlify.toml` :

```
[build]
  command = "npm run build"
  publish = "dist"
  functions = "netlify/functions"

[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200

[[headers]]
  for = "/*"
  [headers.values]
    X-Frame-Options = "DENY"
    X-Content-Type-Options = "nosniff"
    X-XSS-Protection = "1; mode=block"
    Strict-Transport-Security = "max-age=31536000; includeSubDomains"
    Referrer-Policy = "strict-origin-when-cross-origin"

[[headers]]
  for = "/assets/*"
  [headers.values]
    Cache-Control = "public, max-age=31536000, immutable"
```

1. Deploy

```
# Deploy to preview
netlify deploy

# Deploy to production
netlify deploy --prod
```

1. Set Environment Variables

```
# Via CLI
netlify env:set VITE_SUPABASE_URL "https://..."
netlify env:set VITE_SUPABASE_ANON_KEY "eyJ..."

# Or via Netlify Dashboard:
# Site Settings > Environment Variables
```

Option 3: Self-Hosted (Nginx)

Requirements

- Ubuntu 22.04+ or similar

- Nginx 1.18+
- SSL certificate (Let's Encrypt recommended)

Deployment Steps

1. Build Application

```
npm run build
```

1. Upload Build

```
# Upload dist folder to server  
scp -r dist/* user@your-server:/var/www/morphoscan/
```

1. Configure Nginx

Create `/etc/nginx/sites-available/morphoscan` :

```

server {
    listen 80;
    server_name morphoscan.com www.morphoscan.com;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name morphoscan.com www.morphoscan.com;

    ssl_certificate /etc/letsencrypt/live/morphoscan.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/morphoscan.com/privkey.pem;

    root /var/www/morphoscan;
    index index.html;

    # Security headers
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;

    # Gzip compression
    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml application/xml application/xml+rss text/javascript;

    location / {
        try_files $uri $uri/ /index.html;
    }

    # Cache static assets
    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Service worker should not be cached
    location /sw.js {
        expires -1;
        add_header Cache-Control "no-store, no-cache, must-revalidate, proxy-revalidate, max-age=0";
    }
}

```

1. Enable Site

```

sudo ln -s /etc/nginx/sites-available/morphoscan /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx

```


Database Setup

Running Migrations

Option 1: Supabase CLI (Recommended)

```
# Login to Supabase
supabase login

# Link to your project
supabase link --project-ref your-project-ref

# Push migrations to production
supabase db push

# Verify migrations
supabase db diff --schema public
```

Option 2: SQL Scripts

```
# Connect to production database
psql postgresql://postgres:[YOUR-PASSWORD]@db.your-project.supabase.co:5432/postgres

# Run migrations in order
\i supabase/migrations/001_initial_schema.sql
\i supabase/migrations/002_add_dlc_tables.sql
# ... continue with all 98 migrations
```

Database Migration Checklist

- [] Backup current database
- [] Test migrations on staging
- [] Run migrations during low-traffic period
- [] Verify data integrity after migration
- [] Update database types: `npm run db:types`

Seeding Data (Optional)

```
# Seed with sample data (development only)
npm run seed
```

Edge Functions

Supabase Edge Functions

The application uses Supabase Edge Functions for:

- DLC license validation
- Signed URL generation for DLC assets
- Payment webhooks
- NSFW content filtering

Deploying Edge Functions

```
# Deploy all functions
supabase functions deploy

# Deploy specific function
supabase functions deploy get-dlc-signed-url
supabase functions deploy validate-dlc-license
```

Function Environment Variables

Set secrets for edge functions:

```
# Set Stripe secret key
supabase secrets set STRIPE_SECRET_KEY=sk_live_...

# Set DLC encryption keys
supabase secrets set DLC_MASTER_KEY=...
```

Testing Edge Functions

```
# Test locally
supabase functions serve

# Test specific function
curl -X POST \
  http://localhost:54321/functions/v1/get-dlc-signed-url \
  -H "Content-Type: application/json" \
  -d '{"packageId": "nsfw-basic", "assetPath": "images/test.jpg"}'
```

Post-Deployment Verification

Deployment Checklist

1. Application Health

- [] Application loads successfully
- [] No console errors in browser
- [] Service worker registers correctly
- [] PWA install prompt appears (mobile)

2. Core Features

- [] User registration works
- [] User login works
- [] Scanner functionality works
- [] Health diary saves entries
- [] Data syncs with Supabase

3. DLC Features

- [] DLC store loads
- [] Package purchase flow works

- [] License validation works
- [] Content download works
- [] NSFW detection works

4. Security

- [] HTTPS enabled
- [] Security headers present
- [] CSP header configured
- [] No sensitive data in console logs

5. Performance

- [] Lighthouse score > 90
- [] First Contentful Paint < 1.5s
- [] Time to Interactive < 3s
- [] Bundle size optimized (main < 150KB)

Verification Scripts

```
# Check application health
curl -I https://your-domain.com

# Verify security headers
curl -I https://your-domain.com | grep -E "X-Frame-Options|X-Content-Type-Options|
Strict-Transport-Security"

# Check service worker
curl https://your-domain.com/sw.js

# Run Lighthouse audit
npx lighthouse https://your-domain.com --view
```

Testing User Flows

1. Registration Flow

- Create new account
- Verify email
- Complete onboarding

2. Scanner Flow

- Take measurement
- Save scan
- View history

3. DLC Purchase Flow (if applicable)

- Browse DLC store
- Purchase package
- Verify license
- Access content

4. Offline Functionality

- Disconnect internet
- Use app offline
- Reconnect
- Verify sync

Monitoring & Logging

Sentry Integration

Setup

1. Create Sentry project at sentry.io (<https://sentry.io>)
2. Add DSN to environment variables
3. Deploy with Sentry enabled

Monitoring Dashboard

View errors and performance at:

```
https://sentry.io/organizations/your-org/projects/morphoscan/
```

Key Metrics to Monitor

- Error rate
- User sessions
- Performance issues
- Failed API calls
- DLC download failures

Custom Logging

The application includes structured logging via `src/lib/logger.ts` :

```
import { logger } from '@lib/logger';

// Log levels
logger.info('Application started');
logger.warn('Low storage space');
logger.error('Failed to save scan', { error });
logger.debug('Debug information', { data });
```

Supabase Logs

Monitor database activity:

```
# View logs
supabase logs db

# View edge function logs
supabase logs functions
```

Analytics Setup (Optional)

Google Analytics

Add to `index.html` :

```

<!-- Google Analytics -->
<script async src="https://www.googletagmanager.com/gtag/js?id=GA_MEASUREMENT_ID"></script>
<script>
  window.dataLayer = window.dataLayer || [];
  function gtag(){dataLayer.push(arguments);}
  gtag('js', new Date());
  gtag('config', 'GA_MEASUREMENT_ID');
</script>

```

Rollback Procedures

Immediate Rollback

Vercel

```

# List deployments
vercel ls

# Rollback to previous deployment
vercel rollback [deployment-url]

```

Netlify

```

# Via dashboard:
# Deploys > Select previous deploy > Publish deploy

```

Self-Hosted

```

# Revert to previous build
cd /var/www/morphoscan
git checkout HEAD~1
npm run build
# Replace dist files

```

Database Rollback

```

# Revert last migration
supabase db reset --version [previous-version]

# Or manually rollback
supabase db push --dry-run # Preview changes

```

Rollback Checklist

- [] Identify issue and scope
- [] Notify team
- [] Perform rollback
- [] Verify application health
- [] Monitor for issues
- [] Document incident

- [] Plan fix for next deployment

Troubleshooting

Common Issues

1. Build Failures

Problem: Build fails with TypeScript errors

Solution:

```
# Clean install dependencies
rm -rf node_modules package-lock.json
npm install

# Regenerate types
npm run db:types

# Try build again
npm run build
```

2. Environment Variables Not Loading

Problem: Variables undefined in production

Solution:

- Verify variables start with `VITE_`
- Check deployment platform env configuration
- Rebuild after adding variables

3. CORS Errors

Problem: API calls blocked by CORS

Solution:

- Add domain to Supabase allowed origins
- Check Supabase project URL is correct
- Verify API keys are valid

4. Service Worker Issues

Problem: Old version cached

Solution:

```
// Force service worker update
if ('serviceWorker' in navigator) {
  navigator.serviceWorker.getRegistrations().then(registrations => {
    registrations.forEach(reg => reg.unregister());
  });
}
```

5. DLC Content Not Loading

Problem: Content fails to download

Solution:

- Check edge function logs
- Verify license is valid
- Check network tab for 403/404 errors
- Verify Supabase storage buckets exist

6. Payment Processing Failures

Problem: Stripe checkout doesn't work

Solution:

- Verify Stripe publishable key is correct
- Check Stripe webhook is configured
- Verify price IDs match Stripe dashboard
- Check browser console for errors

Debug Mode

Enable debug logging:

```
# Local development
VITE_DEBUG=true npm run dev

# Production debugging (temporary)
localStorage.setItem('debug', 'true');
```

Support Resources

- **Documentation:** See SETUP_GUIDE.md, TESTING_GUIDE.md
- **Supabase Docs:** <https://supabase.com/docs>
- **Vite Docs:** <https://vitejs.dev>
- **Stripe Docs:** <https://stripe.com/docs>

Performance Optimization

Production Checklist

- [x] Code splitting enabled
- [x] Lazy loading for routes
- [x] Images optimized
- [x] Bundle size < 150KB (main chunk)
- [x] Service worker caching
- [x] Gzip/Brotli compression
- [x] CDN enabled
- [x] HTTP/2 enabled

Bundle Analysis

```
# Analyze bundle size
npm run build:analyze

# View report
open dist/stats.html
```

Caching Strategy

- **Static Assets:** 1 year cache
 - **Service Worker:** No cache
 - **API Responses:** Varies by endpoint
 - **DLC Content:** Cache after download
-

Security Best Practices

Pre-Deployment Security Audit

```
# Run vulnerability scan
npm audit

# Check for security issues
npm run scan:vuln

# Verify security headers
curl -I https://your-domain.com
```

Security Checklist

- [] All environment variables secured
- [] No hardcoded secrets
- [] HTTPS enforced
- [] Security headers configured
- [] Input validation enabled
- [] Rate limiting active
- [] Content Security Policy set
- [] Sensitive data encrypted
- [] Database rules configured
- [] Auth tokens expire appropriately

Incident Response Plan

1. **Detection:** Monitor Sentry for security alerts
2. **Assessment:** Evaluate severity and impact
3. **Containment:** Disable affected features if needed
4. **Remediation:** Deploy fix
5. **Notification:** Inform affected users if required
6. **Review:** Post-mortem and prevention

Continuous Integration

GitHub Actions Workflow

Create `.github/workflows/deploy.yml` :

```
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '22'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm run test:run

      - name: Build
        run: npm run build
        env:
          VITE_SUPABASE_URL: ${ secrets.VITE_SUPABASE_URL }
          VITE_SUPABASE_ANON_KEY: ${ secrets.VITE_SUPABASE_ANON_KEY }
          # ... add all required env vars

      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v20
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-org-id: ${ secrets.VERCEL_ORG_ID }
          vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
          vercel-args: '--prod'
```

Final Notes

Post-Deployment Tasks

- [] Update DNS records (if needed)
- [] Configure custom domain
- [] Enable monitoring/alerts
- [] Document deployment in changelog
- [] Notify team of deployment

- [] Monitor application for 24 hours

Maintenance Schedule

- **Daily:** Monitor error logs
- **Weekly:** Review performance metrics
- **Monthly:** Update dependencies
- **Quarterly:** Security audit

Last Updated: Phase 6 - December 2025

Maintained By: MorphoScan Pro Development Team

For deployment support, contact: devops@morphoscan.com